

Q1: Technical Debt Diagnosis

- (a) **Entanglement (CACE):** changing the delivery-time rounding logic shifted the whole input distribution, which is why the unrelated "favorite restaurants" feature got worse too.
- (b) **Undeclared Consumers:** marketing had been reading the raw output table with no API contract, and nobody on the ML team knew, so a future retrain could silently break their campaign.
- (c) **Config/Glue-Code Debt (Pipeline Jungle):** 14 undocumented shell scripts with no orchestration tool means no single source of truth for run order or configuration, so silent failures are a matter of time.

Mitigation for (c): pin a single `environment.yml` so the environment is reproducible, swap the loose scripts for an orchestrated `MLproject` or `dvc.yaml` where each stage has clearly declared inputs and outputs, and track everything centrally with `mlflow.log_param()`, `log_metric()`, and `set_tag("git_commit", ...)`.

Q2: MLflow Experiment Comparison

I trained an MLP on MNIST across six runs, changing the hidden layer sizes, learning rate, and batch size each time to see what actually moved the needle. `mlp-wide-256-128` (0.001 lr, batch 128) came out on top at **97.21%** val accuracy and 0.9719 F1, probably because the wider two-layer network had more room to pick up stroke and edge patterns than the single-layer baseline. There's no sign of overfitting across the 15 epochs either: train loss dropped from 0.2025 to 0.0069 while val accuracy climbed from 94.02% to 97.21% alongside it, with no plateau or reversal. Learning rate mattered the most of everything I tuned. `lr=0.0001` underfit badly (94.02%), `lr=0.01` got a bit unstable (95.93%), and 0.001 landed right in the sweet spot. Architecture width helped a little more on top of that (+0.36%), but shrinking the batch size to 64 barely moved accuracy (96.89%) while nearly doubling training time from 24.1s to 40.3s, so that trade-off wasn't worth it.

Run	Layers	LR	Batch	Train Loss	Val Acc / F1
<code>mlp-wide-256-128</code>	(256,128)	0.001	128	0.0069	0.9721 / 0.9719
<code>mlp-deep-128-64</code>	(128,64)	0.001	128	0.0080	0.9705 / 0.9703
<code>mlp-small-batch-64</code>	(128,)	0.001	64	0.0107	0.9689 / 0.9687
<code>mlp-baseline-128</code>	(128,)	0.001	128	0.0229	0.9685 / 0.9683
<code>mlp-high-lr-0.01</code>	(128,)	0.010	128	0.0344	0.9593 / 0.9593
<code>mlp-low-lr-0.0001</code>	(128,)	0.0001	128	0.2025	0.9402 / 0.9397

```
mlflow.log_param("hidden_layer_sizes", str(hidden_layer_sizes))
mlflow.log_param("learning_rate_init", lr)
mlflow.log_param("batch_size", batch_size)
mlflow.log_param("max_iter", max_iter)
mlflow.log_param("solver", "adam")
mlflow.log_param("random_state", 42)
mlflow.log_metric("train_loss", train_loss)
mlflow.log_metric("val_accuracy", acc)
mlflow.log_metric("val_f1_macro", f1)
mlflow.log_metric("train_time_sec", train_time)
mlflow.set_tag("dataset", "MNIST")
mlflow.sklearn.log_model(model, name="model", serialization_format="cloudpickle")
```

Q3: DVC Data Versioning & Rollback

I initialized DVC in the repo and pushed v1 (1800 rows) to an SSH remote (`ssh://trader@localhost/home/trader/dvc_storage`), then added the rows from `new_labels.zip` to build v2 (2801 rows), re-ran `dvc add`, and committed that with a clear message. To roll back I ran `git checkout <v1-commit> -- data.csv.dvc` followed by `dvc checkout`, which restored v1 exactly, confirmed below:

```
$ git log --oneline --decorate -8      # v1 and v2 are tagged
07cb6f1 (tag: v2) update dataset to v2 2801 lines
8f985a8 (tag: v1) add v1 dataset 1800 rows
$ wc -l data.csv                      # currently on v2
2801 data.csv
$ git checkout v1 -- data.csv.dvc && dvc checkout
M      data.csv
$ wc -l data.csv                      # rolled back to v1
1801 data.csv                        # = 1800 rows + 1 header line
```

Q4: End-to-End Reproducibility Drill

My partner is Prabhav (DA24B018), and we work in a shared private repo holding only the Q4 materials: <https://github.com/Rinkesh-1612/DA3408-Assignment1-Q4>. As Partner A, I trained a RandomForest classifier and logged the params, metrics, seed, a `git_commit` tag, and the model artifact to MLflow, then versioned the dataset with DVC and committed the training script together with the `.dvc` file in a single commit (`409eef7`). I registered the model as `q4-digits-classifier` and moved v1 to **Staging**, giving `val_accuracy` = 0.9639. Prabhav reproduces it from the `partner-a-baseline` tag using only `git clone`, `git checkout`, `dvc checkout`, and `mamba env create -f environment.yml`, then reruns the script and logs a note recording whether the metric matched within tolerance.